PROGRESS LOG

UGV Part 1

ESP32 + L293D rover → PWM speed control, driven over Bluetooth from the Dabble app

Nethra Gupta ♥ **Sara Tantarpale**

ee26btech11219 · ee26btech11227

We used the reference repo github.com/gadepall/voice-ugv and built upon that and made a couple of modifications.

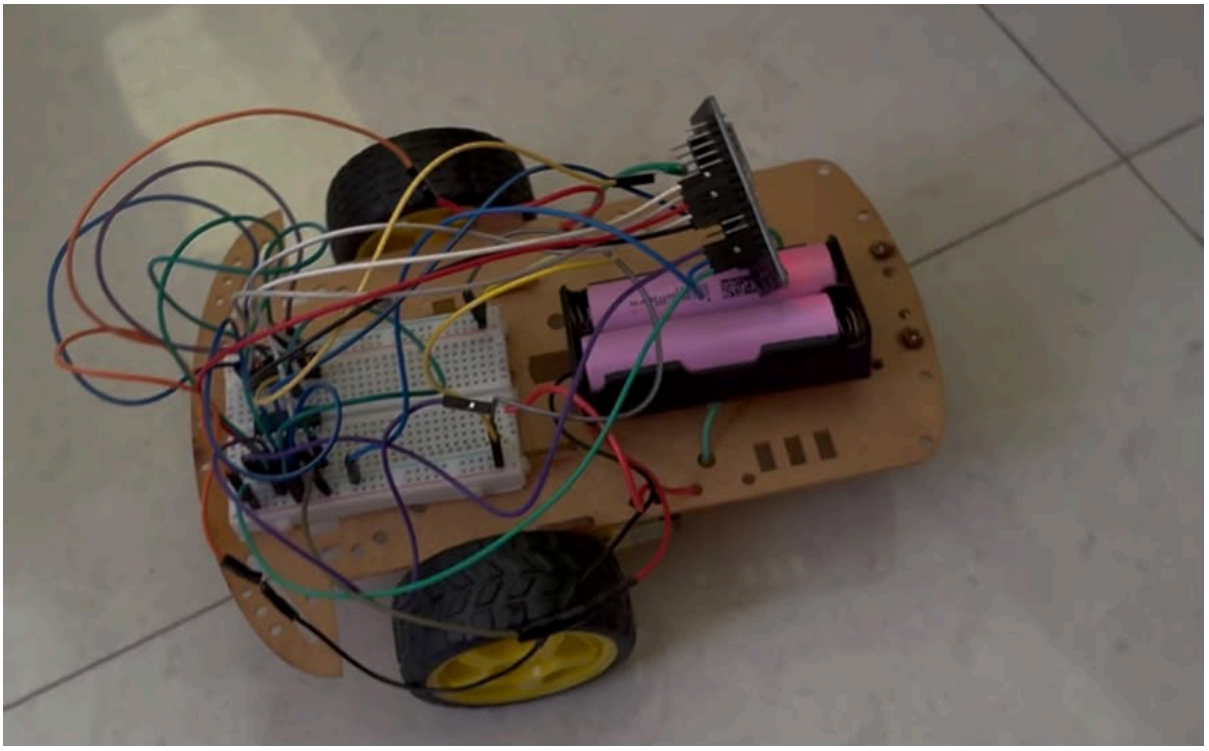


Figure 1. *The finished rover*

01 Components We Used

#	COMPONENT	QTY	USED FOR
1	2WD acrylic chassis	1	Base that holds everything
2	TT gear DC motors (yellow)	2	Drive the wheels
3	Wheels (rubber tyre)	2	Mounted on the motors
4	Caster wheel (swivel)	1	Third wheel for balance, smooth turning
5	ESP32 DevKit (30-pin, ESP-WROOM-32)	1	Microcontroller + Bluetooth
6	L293D motor driver IC	1	H-bridge that drives both motors
7	Mini breadboard (400-point)	1	Holds the L293D and connections
8	18650 Li-ion cells	2	Power, 7.4 V in series
9	2×18650 battery holder (red + black leads)	1	Holds the cells
10	Male-to-female jumper wires	8	ESP32 pins → breadboard
11	Male-to-male jumper wires	about 10	Connections on the breadboard + battery leads
12	Micro-USB data cable	1	Uploading code to the ESP32

#	COMPONENT	QTY	USED FOR
13	Screws / nuts	–	Mounting the motors and caster
14	Android phone with the Dabble app	1	Bluetooth controller
15	Laptop with Arduino IDE	1	Writing and uploading the code

02 What We Did Differently

♥ Added speed control (PWM)

REPO The L293D's speed pins (EN1, EN2) were supposed to be tied to 3.3 V, so the motors were always at full speed.

OURS We connected them to **D27** (left motor) and **D13** (right motor).

WHY The ESP32 can now switch those pins on and off very fast (PWM). How long they stay "on" sets the motor speed.

♥ Smooth steering instead of fixed moves

REPO There were only 5 fixed moves (forward, back, left, right, stop), and turns ran for a fixed 0.6 seconds.

OURS Each wheel gets its own speed, so the car can go straight, curve gently, or spin on the spot.

WHY This is how a 2-wheel car steers (differential drive), and it gives much better control.

♥ Added 3 speed gears

REPO This ties into how we added the PWM. According to the repo, there were no speed options and the motors always ran at full speed.

OURS Slow, medium and fast (40 %, 70 %, 100 %), changed with △ and ✕.

WHY This makes it easier to drive carefully, and a minimum speed is built in so the motors don't just hum without moving.

♥ Motors wired in mirror image

The two motors were mounted facing opposite directions on the chassis. If both were wired the same way, the left wheel would roll forward and the right wheel backward, so pressing ▲ would make the car spin in circles instead of going straight.

FIX We reversed the right motor's wires compared to the left.

- ♡ Left motor: red → **j8**, green → **j5**

- ♡ Right motor: red → **a8**, green → **a5** (swapped)

WHY WIRES The code has INVERT_LEFT / INVERT_RIGHT flags that can flip a motor in software. But swapping two wires on the breadboard is quicker and doesn't need a new upload, so we left both flags as false.

03 Our Final Connections

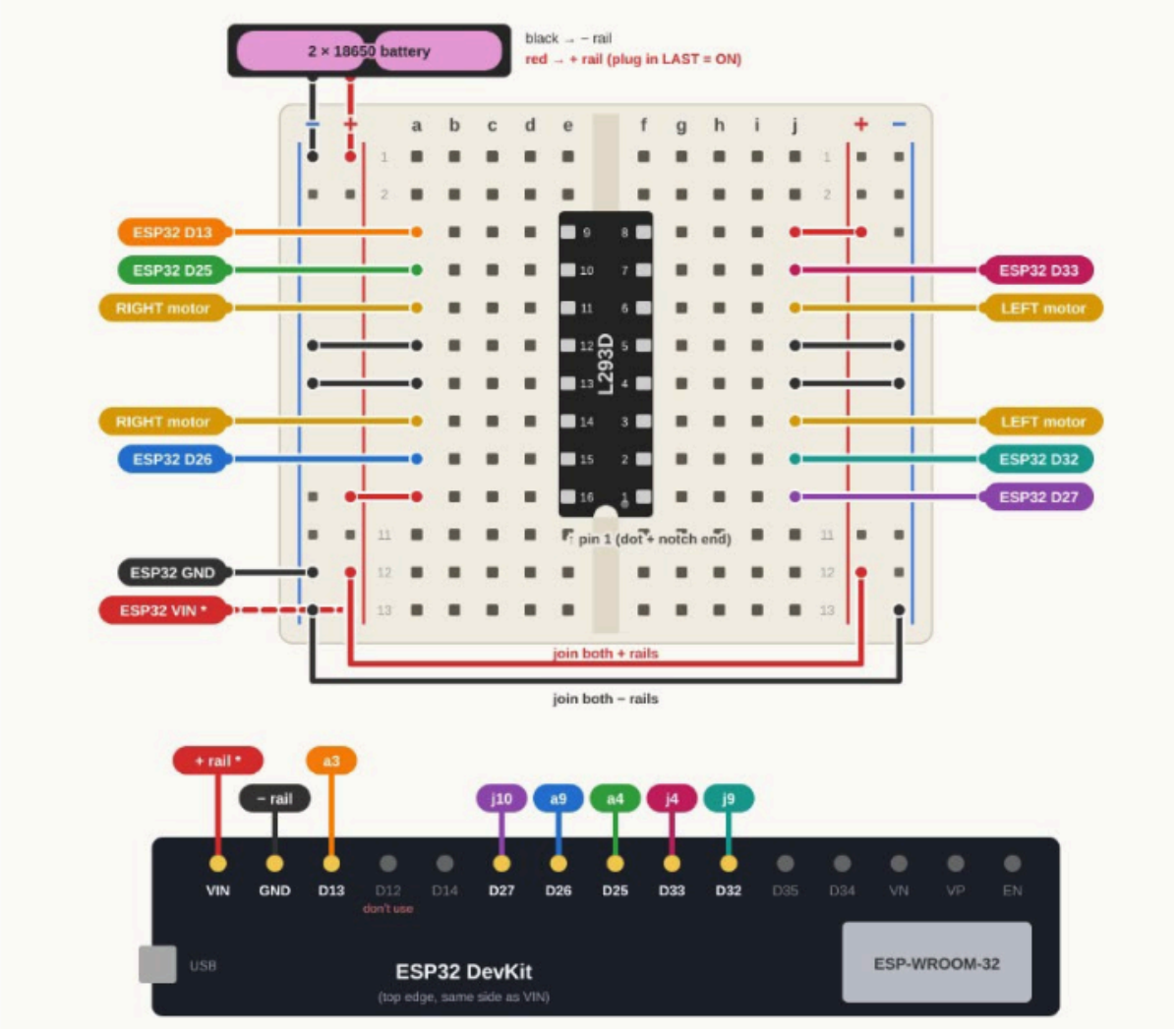


Figure 2. *Wiring diagram: matching colour labels are joined by one jumper wire*

> *We ended up just sticking the battery and the breadboard to the chassis with double-sided tape.*

04 Why Was the ESP32 Heating Up?

The ESP32 board has a small built-in voltage regulator that turns the VIN input into the 3.3 V the chip needs. A regulator like this (a linear one) cannot convert voltage efficiently. It burns the extra voltage as heat.

- The battery gives 7.4 V, and up to about 8.4 V when fully charged.
- The ESP32 needs 3.3 V, so the regulator has to burn off about 4 to 5 V.
- With Bluetooth on, the ESP32 draws roughly 100 to 250 mA.

That is a lot of heat for a tiny regulator with no heatsink, so it gets hot. A 5 V supply would burn only about 1.7 V × 0.2 A ≈ 0.3 W.

> The **L293D** also gets warm, but for a different reason. Its internal switches drop about 2 to 3 V while carrying motor current, and that voltage is also turned into heat. The heat rises when the motors are loaded or stalled.

05 The Buck Converter Question

We thought about using a buck converter like we did for the mobile charger.

> *(The buck converter is a module that efficiently steps 7.4 V down to 5 V with almost no heat.)*

♥ How would using a buck converter be useful?

- ♥ **Much less heat.** The ESP32's regulator now drops only 5 → 3.3 V instead of 7.4 → 3.3 V. From the earlier numbers, heat falls from about 0.82 W to about 0.34 W (0.2 A × 1.7 V).
- ♥ **Steadier supply for the L293D.** Pin 16 would get a steady 5 V instead of anything from about 6.5 V to 8.4 V as the battery drains.
- ♥ **Steadier voltage.** The converter holds 5 V even as the battery drains from 8.4 V down to about 6.5 V. This would help prevent ESP32 resets when the motors start.
- ♥ **Slightly longer battery life.** The ESP32 side draws roughly 1.1 W from the battery instead of about 1.5 W. That saving is small, because the motors use most of the energy, so heat and safety are the main reasons.
- ♥ **Safer USB uploads.** The 5 V rail is the normal level for the ESP32, so it is less stressful for the board.

♥ Why we skipped the buck converter

- **The voltage was already right.** The L293D drops about 2–3 V, so the 7.4 V battery gives the motors about 5 V, within their 3–6 V range.
- **The parts handle 7.4 V.** The L293D accepts up to 36 V, and the ESP32's VIN pin has its own regulator.
- **PWM already controls speed**, so we didn't need a lower voltage.
- **It worked without one.** There were no ESP32 resets, and the regulator only got slightly warm.

(Plus we didn't have one 🥀)

06 How It Actually Works

1 The phone (Dabble)

When you press ▲ or push the joystick in Dabble, the phone sends that over **Bluetooth** to the ESP32. It's like a walkie-talkie message: "forward, 70 % speed".

2 The ESP32

The ESP32 receives the message and works out what each wheel should do. It checks for new commands **50 times a second**.

The car has no steering wheel. It turns by **running its two wheels at different speeds**, like a tank or a wheelchair:

- Both wheels forward at the same speed → **straight**
- Left wheel faster than right → **curves right**
- Left forward, right backward → **spins on the spot**

This is the **differential drive**.

3 The L293D chip

The ESP32 is a tiny brain. Its pins give out only 3.3 V and a few milliamps, far too weak to spin a motor. The L293D takes the

ESP32's small signals and uses them to switch the **big battery power** to the motors.

For each motor, the ESP32 sends the L293D two kinds of instructions:

- **Which direction** (IN pins: D32/D33 for the left motor, D25/D26 for the right). IN1 high and IN2 low makes the motor spin one way. Flip them and it spins the other way, because the chip reverses the current through the motor.
- **How fast** (EN pins: D27 for the left motor, D13 for the right). This is where **PWM** comes in.

4 PWM: how speed control works

The ESP32 can't output "half voltage". It can only switch a pin fully **on** or fully **off**. So it switches it on and off **1000 times a second**:

- On all the time → full speed
- On half the time, off half the time → about half speed
- On a quarter of the time → slow

It's like flicking a light switch so fast you can't see the flicker: the light just looks dimmer. The motor can't react that fast either, so it simply runs at the **average** power. That fraction of "on time" is the **duty cycle**, and the gears (△/✕) change it.

5 The motors

Two small geared motors turn the wheels. The caster wheel just rolls along for balance.

6 The 2×18650 battery

The two cells together give about **7.4 V**. They power the motors through the L293D, and also the ESP32 through its **VIN** pin when the laptop isn't connected. **GND** is the shared return path, so every part's ground has to be connected together.

♥ Two extras in the code

- **Smooth start:** speed changes a little at a time, so the car doesn't jerk and the sudden power draw doesn't reset the ESP32.
- **Safety stop:** if your phone disconnects, the motors stop straight away, so the car doesn't run off on its own.

♥ The whole chain

> ***Phone button → Bluetooth → ESP32 decides each wheel's speed and direction → L293D switches battery power to the motors (direction via IN pins, speed via PWM on EN pins) → wheels turn → car moves.***

07 Our Code

PATH TO CODE `code.ino`

The full Arduino sketch for the ESP32. The important lines from it are explained below.

1 Setting up PWM on the speed pins

```
ledcAttach(pin, PWM_FREQ, PWM_RES);
```

Turns D27 and D13 into PWM outputs at 1000 Hz with 8-bit resolution (duty 0–255).

2 Direction

```
digitalWrite(inA, speed > 0 ? HIGH : LOW);
digitalWrite(inB, speed > 0 ? LOW : HIGH);
```

Sets the motor direction. HIGH/LOW spins it forward and LOW/HIGH spins it in reverse.

3 Speed

```
int duty = MIN_DUTY + (int)(fabsf(speed) * (MAX_DUTY - MIN_DUTY));
pwmWrite(en, ch, constrain(duty, 0, MAX_DUTY));
```

Turns the speed (0 to 1) into a PWM duty between 110 and 255 and sends it to the EN pin. A higher duty means a faster motor.

4 Differential drive (steering)

```
tL = y + x;
tR = y - x;
```

Mixes the joystick's forward value (y) and sideways value (x) into a separate speed for each wheel. Different wheel speeds make the car turn.

5 Gear change

```
if (tri && !prevTri && gear < NUM_GEARS - 1) gear++;
if (cross && !prevCross && gear > 0) gear--;
```

△ raises the gear and ✕ lowers it, once per press (40 %, 70 %, 100 %).

6 Smooth acceleration

```
curL = ramp(curL, tL);
curR = ramp(curR, tR);
```

Moves each wheel's speed toward its target a little at a time (at most 0.08 every 20 ms), so the car doesn't jerk.

7 Safety stop

```
if (!isDeviceConnected) {
  curL = curR = 0;
  drive(0, 0);
  return;
}
```

If the phone disconnects from Bluetooth, both motors stop immediately.

08 Final Notes

- **Mounting:** battery and breadboard stuck to the chassis with double-sided tape.
- **Bluetooth range:** 10–20 meters.

Video of it working

▶ youtu.be/bBGfptQ4Lws